THOMAS P. TRAPPENBERG

Fundamentals of Machine Learning



OXFORD

Fundamentals of Machine Learning

Fundamentals of Machine Learning

Thomas P. Trappenberg
Dalhousie University

OXFORD
UNIVERSITY PRESS

OXFORD
UNIVERSITY PRESS

Great Clarendon Street, Oxford, OX2 6DP,
United Kingdom

Oxford University Press is a department of the University of Oxford.
It furthers the University's objective of excellence in research, scholarship,
and education by publishing worldwide. Oxford is a registered trade mark of
Oxford University Press in the UK and in certain other countries

© Oxford University Press 2020

The moral rights of the author have been asserted

First Edition published in 2020

Impression: 1

All rights reserved. No part of this publication may be reproduced, stored in
a retrieval system, or transmitted, in any form or by any means, without the
prior permission in writing of Oxford University Press, or as expressly permitted
by law, by licence or under terms agreed with the appropriate reprographics
rights organization. Enquiries concerning reproduction outside the scope of the
above should be sent to the Rights Department, Oxford University Press, at the
address above

You must not circulate this work in any other form
and you must impose this same condition on any acquirer

Published in the United States of America by Oxford University Press
198 Madison Avenue, New York, NY 10016, United States of America

British Library Cataloguing in Publication Data

Data available

Library of Congress Control Number: 2019945424

ISBN 978–0–19–882804–4

DOI: 10.1093/oso/9780198828044.001.0001

Printed and bound by
CPI Group (UK) Ltd, Croydon, CR0 4YY

Links to third party websites are provided by Oxford in good faith and
for information only. Oxford disclaims any responsibility for the materials
contained in any third party website referenced in this work.

Acknowledgements

The material in this book has been inspired by several great resources. I would specifically like to acknowledge the influence of Andrew Ng's lecture notes for several parts in this book, especially some basic Bayesian formulations and examples. The excellent book of Francois Cholett on deep learning with Python is very much recommended for both the wonderful explanation of deep learning techniques and the implementation in Keras. We follow much of the presentation of automatic differentiation by the review of Baydin, Pearlmutter, Radul, and Siskind (JMLR 2018) which we recommend for further insights.

Many colleagues have contributed considerably to this book. In particular, I would thank Paul Hollensen, Patrick Connor, and Hossein Parvar for a lot of help with some very rough initial drafts of the manuscript. Thanks to Aditi Nair for her careful reading and good questions, and to Justin Tam for pointing out some rough sections. I would like to thank Will Stone for good ideas and producing Figure 1.7B, and Evangelos Milios for further suggestions. A very special thanks to Farzaneh Sheikhnezhad Fard. Her clear implementation of basic concepts and discussions of deep reinforcement learning have shaped much of the corresponding chapter. Finally, a very special thanks to all my students who took my classes over the last several years and have challenged me to think deeper about machine learning, and to investigate the roots of assumptions that we make.

Preface

Machine learning is exploding, both in research and industrial applications. Although much of the machine learning ideas have been around for many years, the latest breakthroughs are based on several advances. One is the availability of large datasets with labeled data. Another is the availability of fast specialized processors such as graphics processing units (GPUs). In addition, progress is fueled by a deeper understanding of building models and learning from data, as well as some new techniques that brought everything together.

There are now a variety of wonderful books and online resources available on machine learning. So why another book? There are several reasons why I felt compelled to offer my contribution here. Many recent books focus on specific aspects of machine learning, in particular deep learning on the one hand and Bayesian methods on the other. In this book I try to develop a bridge or mutual understanding of what often seems to be viewed as two opposite ends of machine learning. I would like to argue that both approaches are important, have specific strengths in specific application areas, and that a combined view of machine learning and scientific modeling is useful. While this book places some focus on general machine learning methods, I believe that the insight and rigor of probabilistic modeling approaches aid to the general understanding, which in turn offers help in applying machine learning techniques more efficiently.

Another reason that I hope this book is appreciated is that I like to keep explanations brief while still providing some ideas about the deeper reasoning about the methods. It is important to keep this style of the book in mind as treatments and examples are deliberately minimal by design. Also, most explanations are deliberately brief in contrast to more traditional teaching books. My hope is to motivate and guide the reader sufficiently enough to consult further resources for advanced studies. I find this particularly important in an age where there are wonderful resources available on the Internet. I do not claim to cover all details of machine learning, but my hope is to provide the fundamentals for a good understanding that can help to guide further studies.

This book tries to strive a balance between the rigor of mathematical arguments and general outlining principle ideas. In this book, I use mathematical notation mainly as descriptors to keep presentations brief and to show the general form of some equations. For the most part, this book does not include rigorous mathematical proofs or derivations, but I hope to give enough details to see how results can be derived. I know that some readers might tend to avoid mathematical notations, but I would like to encourage these individuals to see them as providing a short form of a story. by contrast, other readers might find my simplifications debatable in a strict mathematical context. However, I think mathematical tools are useful at the level intended here to communicate ideas.

This book includes a brief overview of some older machine learning techniques such as support vector machines and decision trees. While these approaches might

be considered shallow or old-fashioned with respect to deep learning, they have still important practical applications as they might provide solutions to applications that do not require the increased complexity of deep models. We will not dwell for very long into the theory of these traditional methods even though some of the stated formulas seem complex. However, I hope that mentioning some of these ideas, such as kernel methods or Lagrange methods for optimizations with constraints, will add to the foundation of studying more theoretical aspects that are often assumed in modern research papers. This is particularly the case for support vector machines for which there is a rich theory.

Since I have a personal interest in how the brain works, I did include some comments on the relations of machine learning and the brain. The brain is often quoted as inspiration for machine learning methods like neural networks. On the other hand, machine learning is also inspirational for neuroscience by giving us some ideas of possible information processing principles that could be at work in the brain, or at highlighting differences.

In the first chapter we will tour the main ideas of machine learning in order to see where this journey will take us. The next three chapters are designed to apply some machine learning methods. I decided to begin with applications before going through the more rigorous background since applying machine learning methods is not difficult with high-level programming to implement and use a variety of models. These chapters are intended to get you started to run some experiments on your own, and to gain some experience of what we want to achieve with machine learning. Chapter 2 is a brief outline of programming with Python of the kind we need in this book, and Chapters 3 and 4 show how to use sklearn and Keras to implement some of the methods.

The second part of the book comprising the following four chapters is intended to take a deeper look into the foundations of machine learning and scientific modeling in general. This includes a formalization of regression and gradient descent optimization, and discussions of the probabilistic aspects in modeling. The final section of the book comprises the last three chapters which are dedicated to three important and hopefully interesting advanced aspects of machine learning. The first is recurrent neural networks, which capture temporal aspects in modeling; the second is reinforcement learning, which captures learning of agents and is hence a much more general setting of learning machines; and the last chapter consists of some brief thoughts on the impact of machine learning on our society.

Contents

1 Introduction **1**

1.1 The basic idea and history of machine learning 1

1.2 Mathematical formulation of the basic learning problem 5

1.3 Non-linear regression in high-dimensions 9

1.4 Recent advances 12

1.5 No free lunch, but worth the bite 13

I A PRACTICAL GUIDE TO MACHINE LEARNING

2 Scientific programming with Python **17**

2.1 Programming environment 17

2.2 Basic language elements 19

2.3 Code efficiency and vectorization 26

2.4 Data handling 28

2.5 Image processing and convolutional filters 32

3 Machine learning with sklearn **38**

3.1 Classification with support vector machines, random forests, and multilayer perceptrons 39

3.2 Performance measures and evaluations 41

3.3 Data handling 44

3.4 Dimensionality reduction, feature selection, and t-SNE 49

3.5 Decision trees and random forests 52

3.6 Support vector machines (SVM) 55

4 Neural networks and Keras **66**

4.1 Neurons and the threshold perceptron 66

4.2 Multilayer perceptron (MLP) and Keras 68

4.3 Representational learning 73

4.4 Convolutional neural Networks (CNNs) 76

4.5 What and where 85

4.6 More tricks of the trade 86

II FOUNDATIONS
REGRESSION AND PROBABILISTIC MODELING

5 Regression and optimization 93
5.1 Linear regression and gradient descent 93
5.2 Error surface and challenges for gradient descent 96
5.3 Advanced gradient optimization (learning) 98
5.4 Regularization: ridge regression and LASSO 101
5.5 Non-linear regression 105
5.6 Backpropagation 107
5.7 Automatic differentiation 117

6 Basic probability theory 121
6.1 Random numbers and their probability (density) function 121
6.2 Moments: mean, variance, etc. 124
6.3 Examples of probability (density) functions 127
6.4 Some advanced concepts 129
6.5 Density functions of multiple random variables 132
6.6 How to combine prior knowledge with new evidence 134

7 Probabilistic regression and Bayes nets 141
7.1 Probabilistic models 141
7.2 Learning in probabilistic models: Maximum likelihood estimate 144
7.3 Probabilistic classification 147
7.4 Maximum a posteriori (MAP) and regularization with priors 150
7.5 Bayes nets: multivariate causal modeling 153
7.6 Probabilistic and stochastic neural networks 157

8 Generative models 162
8.1 Modeling classes 162
8.2 Supervised generative models 163
8.3 Naïve Bayes 167
8.4 Self-supervised generative models 170
8.5 Generative neural networks 174

III ADVANCED LEARNING MODELS

9 Cyclic models and recurrent neural networks 183
9.1 Sequence processing 184

9.2 Basic sequence processing with multilayer perceptrons and recurrent neural networks in Keras 187
9.3 Gated recurrent neural networks, natural language processing, and attention 190
9.4 Models with symmetric lateral connections 195

10 Reinforcement learning 206
10.1 Formalization of the problem setting 206
10.2 Model-based reinforcement learning 211
10.3 Model-free reinforcement learning 217
10.4 Deep reinforcement learning 222
10.5 Actors and actor-critics 227
10.6 Reinforcement learning in the brain 230

11 Artificial intelligence, the brain, and our society 233
11.1 Different levels of modeling and the brain 233
11.2 Machine learning and artificial intelligence 236
11.3 The impact of machine learning technology on society 238

Index 243

1 Introduction

This chapter provides a high-level overview of machine learning, in particular of how it is related to building models from data. We start with a basic idea in the historical context and phrase the learning problem in a simple mathematical term as function approximation as well as in a probabilistic context. In contrast to more traditional models we can characterize machine learning as nonlinear regression in high-dimensional spaces. This chapter seeks to point out how diverse sub-areas such as deep learning and Bayesian networks fit into the scheme of things and aims to motivate the further study with some examples of recent progress.

1.1 The basic idea and history of machine learning

Machine learning is literally about building machines, often in software, that can learn to perform specific tasks. Examples of common tasks for machine learning is recognizing objects from digital pictures or predicting the location of a robot or a self-driving car from a variety of sensor measurements. These techniques have contributed largely to a new wave of technologies that are commonly associated with artificial intelligence (AI). This books is dedicated to introducing the fundamentals of this discipline.

The recent importance of machine learning and its rapid development with new industrial applications has been breath taking, and it is beyond the scope of this book to anticipate the multitude of developments that will occur. However, the knowledge of basic ideas behind machine learning, many of which have been around for some time, and their formalization for building probabilistic models to describe data are now important basic skills. Machine learning is about modeling data. Describing data and uncertainty has been the traditional domain of Bayesian statistics and probability theory. In contrast, it seems that many exciting recent techniques come from an area now called deep learning. The specific contribution of this book is its attempt to highlight the relationship between these areas.

We often simply say that we learn from data, but it is useful to realize that data can mean several things. In its most fundamental form, data usual consist of measurements such as intensity of light in a digital camera, the measurement of electric potentials in Electroencephalography (EEG), or the recording of stock-market data. However, what we need for learning is a teacher who provides us with information about what these data should predict. Such information can take many different forms. For example, we might have a form of data that we call labels, such as the identity of objects in a digital photograph. This is exactly the kind of information we need to learn optical object recognition. The teacher provides examples of the desired answers that the student (learner) should learn to predict for novel inputs.

Fundamentals of Machine Learning, Thomas P. Trappenberg, Oxford University Press (2020).
© Oxford University Press. DOI: 10.1093/oso/9780198828044.001.0001

Learning will always involve optimizing an objective function, and we will see that the objective function can easily be formulated with specific examples of the desired answers for a learner. This kind of guidance in a learning algorithms is traditionally called "supervised learning." At the other extreme, we might not have any labels, which has traditionally been called unsupervised learning. However, a teacher still needs to provide some guidance in form of an objective, such as ordering data with certain rules. An example of this is clustering, such as when a teacher specifies a distance measure like the Euclidean distance between feature vectors. We will see that such methods are important for representational learning. Finally, a much more general form of learning is a setting where the teacher provides some guidance but the learner, in addition, has to explore possible actions to find novel solutions. Such learning is formalized in reinforcement learning where the objective functions is a slightly more general form compared to the simpler supervised learning. While we will encounter all these different types of learning in this book, most of the fundamentals of learning theory and building models can be demonstrated in the simplest setting of supervised learning.

In machine learning, we are trying to solve problems with computers without explicitly programming them for a specific tasks. We will still need to program the learning machine, and we often have to make some adjustments of such programs for a specific task. However, such an approach is somewhat more general than coding a specific logic for a specific problem. Programming general learning machines instead of specific solutions to a problem is desirable specifically for tasks that would be difficult to program in an explicit rule-based system. A classic example that we will discuss in some length is that of character recognition, as illustrated in Fig. 1.1; writing a program that can translate a visual representation of a character, say the letter A to the computer-interpretable meaning of this character such as representing this letter as the ASCII string 01000001 which is not easy when considering all the shapes and styles that this character can take.

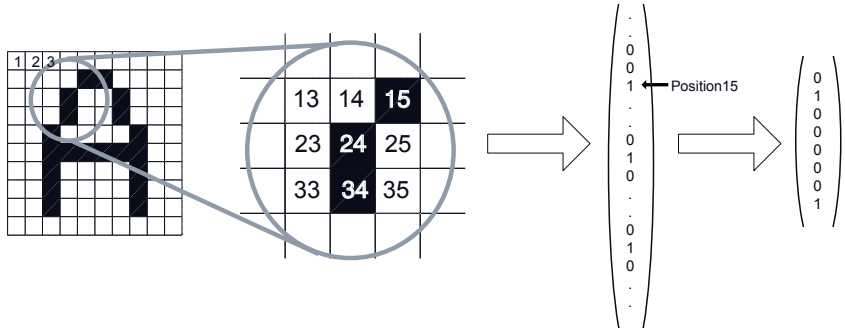


Fig. 1.1 Illustration of a letter-recognition tasks which takes a digital iamge, translates the pixel values into a large vector, and then transform it into a vector that represents the meaning of the letter.

Machine learning might sound like a niche area of science and you might wonder why there is now so much interest in this discipline, both academically and in industry.

The reason is that machine learning is really about modeling data. Modeling is the basis for advanced object recognition, data mining, and, ultimately, intelligent systems. Machine learning is the analytic engine in areas such as data science, big data, data analytics, and, to some extend, to science in general in the sense of building quantitative models.

Machine learning has a long history with traces far back in time (see Fig. 1.2). The first genuine public recognition and accompanying widespread excitement among scientists about learning machines came in the late 1950s and early 1960s with work like Arthur Samuel's self-learning checkers program. Samuel devised a program with reinforcement learning that ultimately learned to outperform its creator. Around this time, Richard Bellman established much of the mathematical foundation of reinforcement learning. One of the first general learning machines that are now considered to be neural networks was invented by Karl Steinbuch in Germany. Frank Rosenblatt invented much of the systematic foundation of neural networks and started to build neural network computers together with Charles Wightman, such as the Mark I Perceptron. Neural networks were popularized again in the 1980s, influenced by David Rumelhart and Geoffrey Hinton, and Terry Sejnowski studied their connection to the brain. Of course, there are many many more inspiring researchers such as Yoshua Bengio, Yann LeCun, and Jürgen Schmidthuber, to name but a few.

We are now in an era of "deep learning", with important recent developments that are responsible for the popularity of machine learning today. This has a great deal to do with the availability of appropriate data and the availability of faster computers, but also to smart techniques that make it possible to scale models to much larger domains. A great example of the recent progress in deep reinforcement learning is the ability of a computer to learn to play video games. Video games from the old Atari platform have become a useful paradigm for a new class of benchmarks that go beyond the classical data sets for machine learning from the University of California Irvine UCI machine learning repository that have dominated the benchmarks in the past. Atari games are somewhat simplified worlds while still presenting more learning in environments that humans have to figure out. In these benchmarks only visual input is given, made up of the computer frames of the video game, and feedback is only provided with how well the player performed in the game.

While these above examples have been widely popularized as the new forefront in AI, much of the scientific progress in machine learning is related to its embedding with probabilistic methods and statistical learning theories. Some pioneers in this domain are Vladislaw Vapnik and Judea Pearl. The development of statistical machine learning and Bayesian networks has influenced the field strongly in the last twenty years, and the domain of Bayesian reasoning is essential for the deeper understanding of machine learning. Some scientists are now working on more general probabilistic programming methods that to some extent go beyond the recent standard in machine learning applications. The aim of this book is to introduce machine learning at a more practical level so that it can be applied immediately by practitioners, at least in its basic form, and then to discuss the foundations in more general terms to help practitioners to learn more about the general theoretical underpinning of machine learning.

In the next three chapters we learn how to apply machine learning with the help of Python-based programming frameworks based on Python libraries such as Sklearn



Fig. 1.2 Some pioneers of machine learning. From top-left to bottom-right: Arthur Samuel playing checkers; Richard Bellman, who formalized reinforcement learning; Karl Steinbuch, who invented the 'learn matrix'; Frank Rosenblatt and Charles Wightman; who implemented a neural computer; Terry Sejnowski and Geoffrey Hinton discussing the Boltzmann machine circa 1983 (Courtesy of Geoffrey Hinton); and David Rummelhardt.

and keras. The next several chapters explore the principle behind supervised learning in the form of regression and classifications. We thereby switch frequently between a functional and a probabilistic framework. A refresher on the basic probability formalism is included in this discussion. In the last few chapters we will discuss some more advanced machine learning issues and methods, including recurrent networks and reinforcement learning.

1.2 Mathematical formulation of the basic learning problem

Much of what is currently most associated with the success of machine learning is supervised learning, sometimes also called predictive learning. The basic task of supervised learning is that of taking a collection of input data $\mathbf{x}$, such as the pixel values of an image, measured medical data, or robotic sensor data, and predicting an output value $\mathbf{y}$ such as the name of an object in an image, the state of a patient's health, or the location of obstacles. It is common that each input has many components, such as many millions of pixel values in an image, and it is useful to collect these values in a mathematical structure such as a vectors (1-dimensional), a matrix (2-dimensional), or a tensor that is the generalization of such structures to higher dimensions. We often refer to machine learning problems as high-dimensional which refers, in this context, to the large number of components in the input structure and not to the dimension of the input tensor.

We use the mathematical terms vector, matrix, and tensor mainly to signify a data structure. In a programming context these are more commonly described as 1-dimensional, 2-dimensional, or higher-dimensional arrays. The difference between arrays and tensors (a vector and matrix are special forms of a tensor) is, however, that the mathematical definitions also include rules on how to calculate with these data structures. This book is not a course on mathematics; we are only users of mathematical notations and methods, and mathematical notation help us to keep the text short while being precise. We follow here a common notation of denoting a vector, matrix, or tensor with **bold-faced** letters, whereas we use regular fonts for scalars. We usually call the input vector a **feature vector** as the components of this are typically a set feature values of an object. The output could also be a multi-dimensional object such as a vector or tensor itself. Mathematically, we can denote the relations between the input and the output as a function

$$y = f(\mathbf{x}). \tag{1.1}$$

We consider the function above as a description of the **true underlying world**, and our task in science or engineering is to find this relation. In the above formula we considered a single output value and several input values for illustration purposes, although we see later that we can extend this readily to multiple output values.

Before proceeding, it is useful to clarify our use of the term "feature." Features represent components that describe the inputs to our learning systems. Feature values are often measured data in machine learning. Sometime the word "attributes" is used instead. In the most part, we use these terms interchangeably. However, sometimes researchers make a small distinction betwen the terms, using attributes to denote unique content while using feature as a derived value, such as the square of an attribute. This strict distinction is usually not crucial for the understanding of the context so our use of the term feature includes attributes.

Returning to the world model in equation 1.1, the challenge for machine learning is to find this function, or at least to approximate it sufficiently. Machine learning offers several approaches to deal with this. One approach that we will predominantly follow is to define a general parameterized function

$$\hat{y} = \hat{f}(\mathbf{x}; \mathbf{w}). \tag{1.2}$$

This formula describes how we make a parameterized hypothesis in which we specify a function $\hat{f}$ that depends on parameters $\mathbf{w}$ to approximate the desired input-output relation. This function is called a **model**:

> *A model is an approximation of a system to study specific aspects of the system and to predict behavior*

This often means that not all of the underlying world has to be captured in depth. For example, a building engineer might make a model of a bridge to tests its static without including the aesthetic aspects that an architect might emphasize in a model. In our context the word model is synonymous with approximation. We have indicated in the formula that this model is an approximation of the desired relation by using a hat symbol above the y and the f. However, we frequently drop the hat symbol when the relation is clear from the context. Also, in the context of machine learning, a model typically includes parameters so that their presence in the notation is synonymous with a model.

Coming up with the right parameterized approximation function is the **hard problem in machine learning**, and we will later discuss several choices. There are methods to develop the approximation function from the data systematically, generally called non-parametric methods. At this point we assume that we have a parameterized approximation function.

We often specify the set of parameters as a vector $\mathbf{w}$ behind a semi-colon in the function arguments. A more appropriate mathematical statement would be that equation 1.2 defines a set of functions in the parameter space. **Learning** is the challenge of finding the values for these parameters that best describe the data, and even better, predicting future outputs y from inputs x. So, the parameters are estimated from data $w = w(x, y)$. Searching for these parameters is done with a learning algorithm. A common way of realizing such a learning algorithm is to define a function that describes the goal of learning, such as minimizing the number of wrong classifications. We call this function the **loss function** $\mathcal{L}$, although other terms are sometimes used in the literature such as objective function, error function, or risk. A common algorithm to minimize such a loss function for a set of given data is to use an algorithm called **gradient descent**, which is an iterative method over the training data that changes the parameters along the negative gradient $\nabla_w \mathcal{L}$ of the loss function,

$$w_i \leftarrow w_i - \alpha \nabla_w \mathcal{L} \tag{1.3}$$

where α is called the learning rate and

$$\nabla_w = \begin{pmatrix} \frac{\partial}{\partial w_1} \\ . \\ . \\ . \\ \frac{\partial}{\partial w_n} \end{pmatrix}. \tag{1.4}$$

is the Nabla operator which signifies the gradient. This is a typical learning algorithm to find the parameters of a model based on example data. We elaborate on this algorithm later.

While the gradient descent can find parameters to minimize the loss of the training data, our real goal is to find the values of $\mathbf{w}$ that best predicts data that have not been

seen before. Just describing the training data acts somewhat more like a memory, but being able to **generalize** is the main goal of machine learning. A good solution of the machine (model) learning problem is represented by a point in the parameter space that approximates best the true underlying world. However, since we usually don't know the true underlying world, we estimate how good this model is by evaluating how good new predictions are.

In some applications of supervised learning we want to predict a continuous output variable. For example, we might want to predict the price of a house from the size information. This is called **regression**. In contrast, sometimes we want to predict discrete values such as the categories of object in a picture. This is called **classification**. The output variable y in classification is called a **label**. It is now common to refer generally refer to the output of the supervised learner as label, even in the case of regression where we have a continuous "label." We will see later that regression and classification are closely related; for example, binary classification can be seen as a regression problems with a discrete function $f(x)$ such as a sign function which would give us two labels, positive and negative.

An important part of our treatment of machine learning is to consider cases with uncertainty. For example, we might not be able to predict an exact label or output value, and we would be best-served if we consider the probability that a certain value will occur. This is very important for several reasons. For example, it is quite common that the process under investigation includes random (stochastic) factors or hidden (latent) factors that create variations in results even for the same sensory states. Thus, we pose that the true underlying world model we seek to describe is better described by a probability density function

$$p(Y = y|\mathbf{x}). \tag{1.5}$$

This function gives the probability density, or probability mass in the case of categorical data, of the label Y having a value y given that we have an input vector $\mathbf{x}$. The arguments of density functions are provided after the vertical bar, and we write random variables as upper-case letters and we specify specific values with lower-case letters.

Formulating machine learning in a probabilistic (stochastic) context has been most useful and provides us with the formalization that created many insights. In the probabilistic framework we are then modeling a density function

$$\hat{p}(Y = y|\mathbf{x}; \mathbf{w}). \tag{1.6}$$

Density function approximation is in some sense a special case of function approximation as the density function is still a function, albeit with some constraints such as a normalization $\int p(y)\mathrm{d}y = 1$. However, modeling density functions is also the more general case for modeling functions in the sense that they proivide the probabilistic information of how likely label values are given a certain input.

A probabilistic framework can be used for a more general formulation of learning. Given a parameterized model as written in equation 1.6, we want to know for all possible parameters how likely they are to describe the data well. This is not necessarily a single solution, and ideally we would like to know the probability density function of the parameters given by the data,

$$p(\mathbf{w}|y, \mathbf{x}). \tag{1.7}$$

This general approach of using probabilistic models and data to estimate model parameters, and in turn make predictions, is the essence of Bayesian modeling. We will introduce this more general learning paradigm. However, many of the current machine learning algorithms use a learning principle where learning only uses the most likely parameters given the data,

$$\mathbf{w}^* = \text{argmax}_w p(\mathbf{w}|y, \mathbf{x}). \tag{1.8}$$

Such a maximum *a posteriori* choice, or some point estimates derived from related principles such as the maximum likelihood estimate, are currently the dominant forms of machine learning. While limited in a Bayesian sense, these approximations have been very useful to build practical applications of machine learning. Of course, at this point we still need to find the specific form of the probability function $p(\mathbf{w}|y, \mathbf{x})$, which in itself is a subject of a hypothesis in the Bayesian framework. The point here is to suggest how useful a probabilistic formalism is in machine learning. We will discuss these thoughts more later in this book.

Formulating specific probabilistic models for problems with many stochastic factors is demanding. An important and useful way to formulate multivariate probabilistic models is the area of **causal models**. Such models provide specific probabilistic models of the components that provide the necessary foundations of the inference engine. Inference here means that the system can be used to "argue" about a solution in a probabilistic sense or to derive predictions. Such systems are the domain of Bayesian networks, and we will include an introduction to this important domain in this book as it provides an important aspect of machine learning. To some extent we will argue that probabilistic models and deep neural networks represent somewhat a diverse spectrum of machine learning models, but we will also argue that they can be viewed in a unified way. Fig. 1.3 shows a famous example form Judea Pearl, one of the inventors of this important modeling framework.

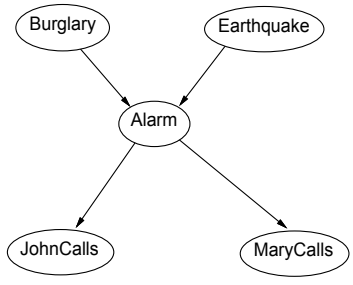


Fig. 1.3 An example of a graphical representation of a causal model.

In Bayesian networks we model entities that build the causal structure of the problem. It hence has a much stronger theoretical predictive strength and explanatory ability than many other machine learning models. However, in practice we rarely know about the generative elements, and machine learning models that are more general such as neural networks represent an often more practical method. Nature might have combined such strategies in the brain, where sensory input is transformed into a semantic space in which some approximation of Bayesian reasoning can be

implemented. Learning about those different aspects of modeling and machine learning is hence a useful approach in this dynamic area.

1.3 Non-linear regression in high-dimensions

The simplest example of supervised machine learning is linear regression. In linear regression we assume a linear model such as the function,

$$y = w_0 + w_1 x. \tag{1.9}$$

This is a low-dimensional example with only a single feature, value x, and a scalar label, value y. Most of us learned in high school to use mean square regression. In this method we choose as values for the offset parameter w_0 and the slope parameter w_1 the values that minimize the summed squared difference between the regressed and the data points. This is illustrated in Fig. 1.4A. We will later explain this procedure in more detail. This is an example where data are used to determine the parameters of a parameterized model, and this model with the fitted parameters can then be used to predict y values for new x values. This is in essence supervised learning.

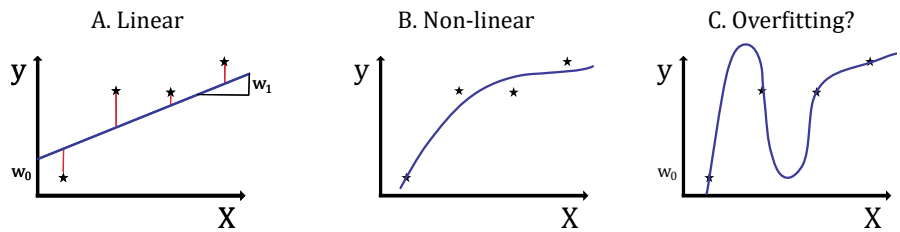


Fig. 1.4 Data points and possible models to fit these data. (A) Linear regression, (B) a non-linear function, and (C) another non-linear function that might overfit the data.

What makes modern machine learning go beyond this type of modeling is that we are now usually describing data in high dimensions (many features) and to use non-linear functions. This seems straight forward, but there are several problems in practice going down this route. For example, Fig. 1.4B shows a non-linear function that seems somewhat to describe the pattern of the data much better than the linear model in Fig. 1.4A. However, the non-linear model shown in Fig. 1.4C is also a solution. It even goes through all the training points. This is a particularly difficult problem. If we are allowed to increase the model complexity arbitrarily, then we can always find a model which goes through all the data points. However, the data points might have a simple relation, such as the linear one of Fig. 1.4A, and the variation only represents noise. Fitting the data point with this noise as in Fig. 1.4C does therefore mean that we are **overfitting** the data.

So a major problem when fitting data with fairly general non-linear functions is the complexity of the function in terms of the number of parameters, such as the order of the polynomial or the number of nodes in neural networks, as discussed further later in

this chapter. We will later discuss methods to prevent overfitting in more detail, but at this point it is already useful to consider some systematics. For example, in the linear model we have some systematic **bias** of the data in that all but the first data point lay above the model curve, and this trend we would also expect to show in new data points if the correct model is the one shown in Fig. 1.4B. In contrast new data points when evaluating with the model on the right would have large variations as the line clearly overshoots in order to hit the training points. This increase of **variance** in the test data is one indication of overfitting. It is hence instructive to study the difference between the error of the training set and the test set as shown in Fig.1.5.

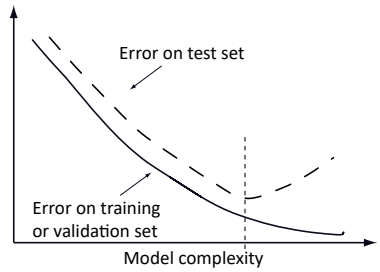


Fig. 1.5 Illustration of bias-variance trade-off. Overfitting shows when the error of the test set is increasing, relative to the training error.

The need to find the right balance between these two effects is called the **bias-variance trade-off**. When the models are too low-dimensional we expect that both errors are high but that both curves stay close to each other. In contrast, when we have plenty of parameters it is possible to make the training error small, while this leads to overfitting and hence an increase in the test error. This bias-variance trade-off is quite important in practical applications of machine learning to guide the developer to the appropriate model choice. There are important techniques under the umbrella term **regularization** with the aim of helping to make the data more regular with respect to the model and hence preventing overfitting.

Building nonlinear models is a challenge in itself as the choices are infinite. We could consider a polynomial of order n, that can be written as

$$y = w_0 + w_1 x + w_2 x^2 + ... + w_n x^n. \tag{1.10}$$

The above function is a function in one dimension. If we have more feature values we could end up with functions that depend on a combination of the feature values such as

$$y = w_0 + w_1 x_1 + w_2 x_2 + w_3 x_1 x_2 + w_4 x_1^2 x_2, ... \tag{1.11}$$

Also, using a polynomial as a non-linear function is only one possible choice of many. We could add trigonometric functions, or some functions which we just make up. We will later consider mainly functions that have been termed artificial neural networks. These functions can be represented graphically, as shown in Fig.1.6. Each node in such a graph is also called a neuron as it resembles somewhat the conjectured basic functionality of a biological neuron in the brain. Such an artificial neuron weights

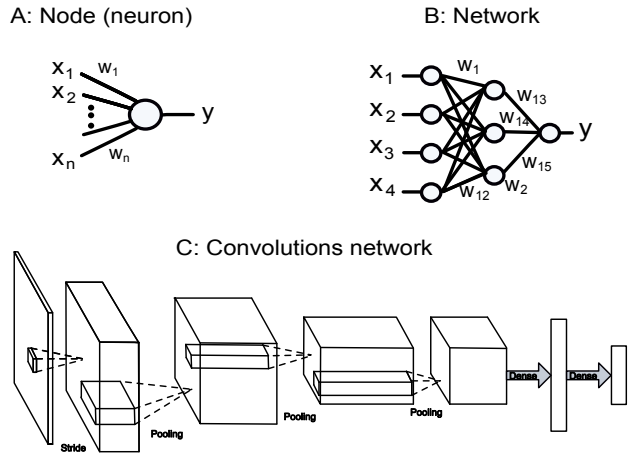


Fig. 1.6 Basic elements of an artificial neural network (ANN). Each node represents an operation of summing weighted inputs and applying an non-linear transfer functions to this net input. The output of each node can become the input of another node or represent the output of the networks.

each individual input with an adjustable parameter, sums this weighted input, and then applies a non-linear function such as a $\tanh$ function on this summed input. The output of each node is hence,

$$y_j = \tanh(\sum_i w_{ji} x_i). \tag{1.12}$$

This output can be the input to another node, and we can in such a way build elaborate functions with graphs of such nodes as shown in Fig.1.6B. We will later elaborate on specific network architectures that represent specific classes of non-linear functions that will be useful for specific applications. We will specifically explore how networks with many layers of neurons have advanced the capabilities of learning machines considerably, which is now known as **deep learning**.

An example of a network type called a convolutional neural network is illustrated in Fig.1.6C. It is now common that such networks have tens and even hundreds of layers. Deep neural networks are hence a form of high-dimensional non-linear fitting function, and preventing overfitting is therefore a very important component in deep learning. Deep networks have many free parameters, and large data sets (big data) have therefore been important for the recent progress in this area, in combination with other techniques to prevent overfitting such as a technique called dropout that we will discuss later. In general, one can think about techniques to prevent overfitting by restricting the possible range of the parameters. While relying on big data is one simple solution, making complex systems work with a limited amount of training data is the much more interesting challenge.

In the next section we will see that basic implementation of machine learning methods is not difficult when using application programs that implement these techniques. This is good news. However, a deeper understanding of the methods is necessary to

make these applications and their conclusion appropriate. The machine learning algorithms will come up with some predictions, and evaluating whether these predictions are sensible is an important part of machine learning. Machine learning education needs therefore to go beyond learning how to run an application program. This book aims to find a balance between practical applications and their theoretical foundation.

1.4 Recent advances

Many advances have been made in recent years based on machine learning, in particular with deep learning methods for image processing, natural language processing, and more general data analytics. Many companies are now enthusiastic about data analytics, using data in a wider sense to gain insights into customer profiles or other data mining tasks. Machine learning is an important part of a data analytics engine. Data analytics often require additional care such as data security to ensure privacy, the ability to acquire and maintain large data collections, and also to make results available in a form useful for humans. We will not delve into many of these aspects but concentrate instead on the data modeling aspects.

One of the most visible impacts of deep learning has been made in computer vision through convolutional neural networks. The basic applications in this area are mostly based on recognition networks and methods for semantic segmentation. However, such methods have now also advanced object localization, object tracking, and scene understanding, to name but a few. Some examples from my own projects are shown in Fig. 1.7. The left-hand image shows semantic segmentation to identify and localize crop and weed for a robotic farming application. The right-hand image shows an application of fish tracking for aquaculture applications.

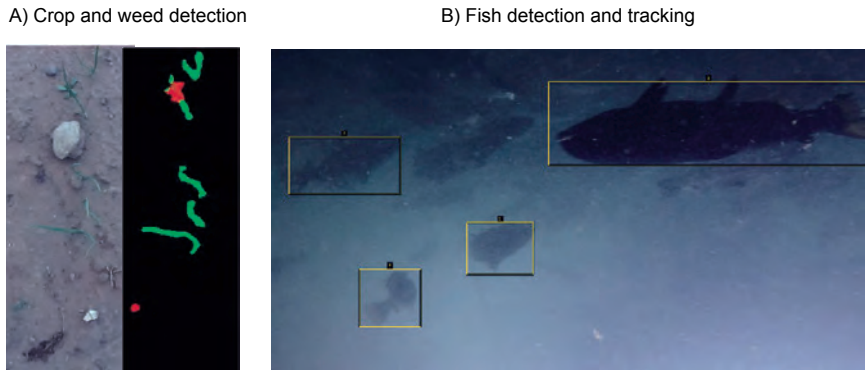


Fig. 1.7 Some examples in computer vision using deep networks. A) Detection of crop (green) and weed (red) to enable automated weeding for farm robotics. The crop is onion in an early growing stage (Courtesy of Nexus Robotics Inc.). B) Recognition, localization, and tracking of fish for fish health monitoring in aquaculture (Courtesy of ReelData Inc.).

Another area that has seen a huge improvement is the area of natural language processing (NLP). It has long been an important tasks to build programs that understand natural languages for applications such as translation, sentiment analysis, or to enable

some form of formal analysis of technical reports. Various methods for sequence modeling have contributed greatly to this area, in particular recurrent neural networks, discussed later in this book.

A developing area in machine learning are generative models. Generative models are models that can make examples of instances of a class. For example, a generative models can learn about cars from examples and then generate images of new cars by itself. Such networks could then be used in some creative way. Examples of systems that can learn generative models are variational autoencoders (VAEs) and generative adversarial networks (GANs). These methods demonstrate an important advance: the ability to capture the probabilistic structure of objects which in turn can be exploited in various ways.

Machine learning methods have shown that it can produce solutions to problems that have previously been intractable. For example, computer programs to play the Chinese board game "Go" have been mostly available only at an advance novice level until a few years ago. However, in 2016, a machine learning program called "Alpha-Go" that combined cleverly supervised and reinforcement learning was able to beat a player, Mr. Lee Sedol, who is considered one of the best players of the last decade and had previously won sixteen world titles. Go was considered to be a real challenge for AI systems as it was considered to rely a lot on "gut feelings" rather than quantifiable strategies. It was therefore a huge success when computers, which had only reached levels of an advanced beginner a few years prior, could win against such an accomplished player.

Finally, AI has always been strongly associated with the understanding of the human mind. While the subject of this book is far from explaining human intelligence, the advances in machine learning have shed light on some aspects of human cognitive and brain processes. For example, generative models with sparseness constraints are able to reproduce the form of representations that brain scientists have measured in the early visual cortex, and there is a lot of evidence, by comparing brain imaging data to deep learning vision systems, that there are some aspects covered by such networks. Bayesian reasoning and reinforcement learning are areas that are now inspiring new approaches in psychiatry. The brain has many aspects that are not captured by current machine learning models, but these models capture some aspects of application domains that have long been difficult to program with traditional computers.

1.5 No free lunch, but worth the bite

Neural networks and other models, such as support vector machines and decision trees, are fairly general models in contrast to Bayesian models that are usually much better at specifying a causal structure of interpretable entities. More specific models should outperform more general model as long as they faithfully represent the underlying structure of the world model. This fact is captured by David Wolpert's "No free lunch" theorem, which states that there is not a single algorithms that covers all applications better than some other algorithms. The best model is, of course, the real world model, as discussed earlier, which we generally do not know. Applying machine learning algorithms is therefore somewhat of an art and requires experience and knowledge of the constraints of the algorithms. Discussions of what is an appropriate model are